Ders 21 Çalışma Özeti

Ders 21 Çalışma Özeti

Genel Konular

- Akış diyagramı ve kod ilişkisi
  - Önce kodu yazıp sonra akış diyagramını çizmek yanlış bir alışkanlıktır. Algoritma tasarlanırken akış diyagramı çizmek faydalıdır. Ancak çok deneyimli olunca bu adım atlanabilir, ilk yılda atlanmamalıdır.
  - Akış diyagramının %80-90'ı tamamlanmış olmalıdır. Kod yazarken ve analiz yaparken gözden kaçan küçük detaylar düzeltilebilir.
  - Akış diyagramında dile özgü durumlar kullanılabilir. Örneğin C'de karakter dizisi null karakterine kadar okunabildiği için akış diyagramında da "null görene kadar" ilerlemek kabul edilir.
  - Matrislerde gösterim `A[i][j]` şeklinde tercih edilir. `A[i, j]` yanlış sayılmaz, ama ilki tercih edilir. Matrisin adı `A`, indisi `a` gibi karışacak adlar kullanılmamalıdır.
- İki sıralı diziyi birleştirme (merge) — öğrenci çözümleri (ekrandan, 00:35:00 ve 01:04:20)
  - Birinci çözüm: `I = 1`, `J = 1`. While: `I < N AND J < M`. `Dizi1[I] > Dizi2[J]` ise `result[K] = Dizi2[J]`, `J = J + 1`. Değilse `result[K] = Dizi1[I]`, `I = I + 1`. Ardından `result` yazdırılır. Hoca bu koşulu kabul etti. (Ders sonrası not: indisler 1'den başladığı için büyük olasılıkla `I <= N` ve `J <= M` olmalıdır.)
  - Bu çözümün eksikleri:
    - K'ya başlangıç değeri verilmemiştir (`K = 1`).
    - K hiç artırılmamaktadır. `K = K + 1` her iki kolun birleştiği yere bir kez yazılmalıdır.
    - Dizilerden biri bitince diğerinde kalan elemanlar `result`'a eklenmemektedir.
  - İkinci çözüm (0'dan başlayan indislerle): `I = 0`, `J = 0`. While: `I < N AND J < M`. `A[I] > B[J]` ise `C[I+J] = B[J]`, `J = J + 1`. Değilse `C[I+J] = A[I]`, `I = I + 1`. Döngüden sonra:
    - `I < N` ise `K ← I || 1 || N−1` ile birinci dizinin kalanı eklenir. Çizimde bu adım `C[I+K] = A[K]` olarak yazılmıştı. Derste üzerinde durulmadı, ama bu noktada J = M olduğu için doğru hedef `C[K+J]`'dir. `C[I+K]` yazılırsa I sabit kaldığı için elemanlar yanlış gözlere yazılır.
    - Değilse `K ← J || 1 || M−1` ile `C[I+K] = B[K]`: ikinci dizinin kalanı eklenir. Burada I = N olduğu için bu yazım doğrudur.
    - Hoca "iki dizi aynı anda bittiyse" durumunu da andı: ikinci for'a hiç girilmez, ayrıca `J < M` kontrolü gerekmez. (Ders sonrası not: her adımda yalnız bir indis ilerlediği için bu durum boş olmayan dizilerde büyük olasılıkla oluşmaz.)
  - `C[I+J]` kullanımı ayrı bir K sayacı gerektirmez. Ancak bu daha verimli değildir: `K + 1` bir işlem, `I + J` (veya 1'den başlayınca `I + J − 1`) bir ya da iki işlemdir.
- Sınır kontrolü yerine diziye "nöbetçi" eleman eklemek (ekrandan, 01:35:10)
  - Fikir: iki dizinin sonuna, dizilerin bütün elemanlarından büyük bir değer eklenirse, bir dizinin bitip bitmediğini ayrıca kontrol etmeye ve kalanları ayrıca eklemeye gerek kalmaz.
  - Hoca 1000 yerine ne yazılabileceğini sorar. (Ders sonrası yorum: sabit bir değer her zaman yeterli olmayabilir, dizilerin elemanlarından türetilmesi gerekir.)
  - "Dizilerden birinin son elemanından bir fazlası" önerisi her durumda çalışmaz. Örneğin bir dizi 13 ile, diğeri 15 ile biterse 14 eklemek yanlıştır.
  - "İki dizinin son elemanlarını karşılaştırıp büyük olanın bir fazlasını eklemek" çalışır, ama bir karşılaştırma gerektirir.
  - Karşılaştırmasız çözüm: iki dizinin son elemanlarının toplamı her iki dizinin sonuna

eklenir. Negatif sayılar varsa mutlak değerlerin toplamı kullanılmalıdır. Örnek: `2 4 6 8 12` ve `1 3` için iki dizinin sonuna 15 eklenir.
  - Döngü koşulu için konuşulan seçenekler:
    * `I <= N+1 AND J <= M+1`: nöbetçi elemanlardan biri de sonuç dizisine yazılır. Sonuç yazdırılırken son eleman atlanmalı veya dizinin boyutu bir azaltılmalıdır.
    * `I ≠ N+1 OR J ≠ M+1`: iki indis de kendi nöbetçisine gelene kadar devam edilir. Nöbetçi elemanlardan hiçbiri yazılmaz. Koşul AND ile yazılırsa, dizilerden biri bittiği anda döngü erken biter ve diğer dizinin kalanı yazılmaz.
- Merge sort'un genel fikri (yalnızca soru üzerine)
  - Sırasız dizi her turda ikiye bölünür, tek eleman kalana kadar bölme sürer. Sonra parçalar sıralı şekilde birleştirilerek yukarı doğru çıkılır. Bu bir böl ve yönet (divide and conquer) yaklaşımıdır ve bu dersin konusu değildir.
  - Derste yalnızca birleştirme (merge) adımı yapıldı. Önemli olan, bu adımın N + M karşılaştırmada tamamlanmasıdır. İki diziyi arka arkaya ekleyip sıralamak çok daha maliyetlidir.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir işlem hem true hem false kolunda yapılacaksa iki kola ayrı ayrı yazmak yerine kolların birleştiği yere bir kez yazın. Örnek: birleştirmede `K = K + 1`.
- `A[I] > B[J]` sorusunun cevabı hayır ise bu `A[I] <= B[J]` demektir. Eşitlik için ayrıca bir kontrol gerekmez. Bir öğrenci eşitlik durumunda hiçbir şey yapılmadığını düşünmüştü: false koluna gidildiği için eşit elemanlar da yerleştirilir.
- Bir değişken azaltmak için büyük çaba harcamayın. Gerçekten gereksiz bir değişken varsa kullanmayın, ama bir değişken az kullanmak bu tür durumlarda özel bir kazanç sağlamaz.
- Nadiren karşılaşılacak bir durum için her seferinde ek kontrol yapmayın. Örneğin "ikinci dizinin ilk elemanı birincinin son elemanından büyükse doğrudan uç uca ekle" kontrolü, 100 seferden belki 7'sinde işe yarar ve 93 kez boşuna yapılır.
- Analizi açık ve düzenli yazın. Kağıda hızlıca karalanan bir analizde hata görülmez. Bu derste AND ile OR'un karıştırılması ancak düzenli analizle ortaya çıktı.
- Sınır değerleri için yalnızca büyük/küçük değil, "eşit değil" koşulu da kullanılabilir. Bu da zaman zaman işe yarayan bir fikirdir.

Kısa Tekrar Notları

- Önce akış diyagramı, sonra kod.
- Birleştirme: `I`, `J`, `K` başlangıç değerleri. Küçük olanı `result[K]`'ya yaz, ilgili indisi artır, `K`'yı kolların birleştiği yerde artır.
- Döngüden sonra hangi dizi bitmediyse kalanını sona ekle.
- `C[I+J]` kullanımı K sayacının yerini tutar ama daha verimli değildir.
- Nöbetçi eleman: iki dizinin sonuna bütün elemanlardan büyük bir değer (ör. son elemanların mutlak değerlerinin toplamı).
- Nöbetçiyle döngü koşulu: `I ≠ N+1 OR J ≠ M+1`.
- Birleştirmenin karmaşıklığı: N + M.

Detaylı Açıklamalar

Önceki derste verilen "iki sıralı diziyi birleştirme" problemi için öğrencilerin gönderdiği çözümler ekranda incelenir. Son bölümde dizilerin sonuna büyük bir değer ekleyerek sınır kontrolünden kurtulma fikri tartışılır ve doğru döngü koşulu bulunur.

Birinci öğrenci çözümünün incelenmesi

Çözümde karşılaştırma doğru yapılıyordu, ama `result[K]`'ya yazılan K hiç başlatılmamış ve artırılmamıştı. Sınıfta önce K'nın tanımlanmadığı söylendi. Hoca sorunun tanımlama değil, başlangıç değeri ve artırma olduğunu belirtti: `K = 1` I ve J ile birlikte atanmalı, `K = K + 1` ise her iki kolun birleştiği yere bir kez yazılmalıdır.

İkinci eksik, dizilerden biri bittiğinde diğerinin kalan elemanlarının sonuca eklenmemesiydi. While'dan çıktıktan sonra `I < N` mi yoksa `J < M` mi olduğuna bakılarak bitmeyen dizinin kalanı sona eklenmelidir.

Bir öğrenci, uzun dizinin son elemanı kısa dizinin son elemanından küçükse kalanları doğrudan eklemenin sırayı bozup bozmayacağını sordu. Örneğin `... 13` ile biten kısa bir dizi ve `... 10 11` ile biten uzun bir dizi. Hoca örnek üzerinde gösterdi: 10 ve 11, 13'ten küçük oldukları için döngü içinde 13'ten önce yerleştirilir. Döngü ikinci dizi bitince sona erer ve yalnızca 13 sona eklenir. Sıra bozulmaz.

Nöbetçi değerin seçilmesi

Hoca, `1 3 5 8 9` ve `2 4 6 7 11 13` dizilerinin sonuna 1000 ekleyerek fikri gösterdi. Döngü bütün gerçek elemanları yerleştirir ve iki nöbetçi karşılaştırılırken durması gerekir. (Ders sonrası yorum: 1000 her zaman yeterince büyük olmayabilir.) Hoca 1000 yerine ne yazılabileceğini sorar.

Önerilenlerden "bir dizinin son elemanından bir fazlası" (ör. 14) yanlıştır: diğer dizi 15 ile biterse 15'ten önce 14 gelir. "Son elemanların büyüğünün bir fazlası" doğrudur ama karşılaştırma gerektirir. Hoca karşılaştırmasız bir yol sordu. Bir öğrenci iki son elemanın toplamını önerdi. Bir başkası, negatif sayılar olabileceği için mutlak değerlerin toplamının kullanılması gerektiğini ekledi.

Döngü koşulu: AND mi, OR mu?

Nöbetçiler eklendikten sonra `I <= N+1 AND J <= M+1` koşuluyla döngü, iki dizinin nöbetçilerinden biri de yazılana kadar devam eder. Bu durumda sonuç dizisinin son elemanı yazdırılmamalıdır.

Bir öğrenci `I ≠ N+1 ... J ≠ M+1` koşulunu önerdi. Hoca bunu önce AND ile yazdı. `2 4 6 8 12` ve `1 3` dizilerinde J, 3'e (yani M+1'e) ulaştığında koşul bozuldu ve döngü 4, 6, 8, 12 yazılmadan bitti. Bunu bir öğrenci fark etti. Öneri aslında OR iledir: `I ≠ N+1 OR J ≠ M+1`. Bu koşulla, iki indis de kendi nöbetçisine gelene kadar devam edilir. I = 6 ve J = 3 olduğunda döngü biter ve hiçbir nöbetçi yazılmaz.

Ders sonrasında bu yöntem 0 ile 7 arasındaki değerlerden oluşan rastgele sıralı dizilerle denendi. Toplam nöbetçi, dizilerden birinin son elemanı 0 olduğunda diğer dizinin son elemanına eşit çıkabiliyor. Bu durumda nöbetçi gerçek bir eleman gibi yazılıyor ve döngü taşıyor. Örnek: `0` ve `0 0 7`. Nöbetçi olarak $|a_N| + |b_M| + 1$ kullanıldığında bütün denemeler doğru sonuç verdi. Derste kullanılan pozitif sayılı örneklerde bu sorun ortaya çıkmaz.

Hoca, aynı problem için konuşulan yolları şöyle özetledi:

1. Döngüden sonra bitmeyen dizinin kalanını sona eklemek.
2. Son elemanların toplamını nöbetçi olarak eklemek, `<= N+1` / `<= M+1` koşuluyla döngüyü kurmak ve sonuçtaki son elemanı yazdırmamak.
3. Nöbetçi eklemek ve `I ≠ N+1 OR J ≠ M+1` koşuluyla döngüyü kurmak.

Her yöntemin kendi artıları ve eksileri vardır. Önemli olan birleştirmenin N + M adımda yapılmasıdır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.